Lahore Air Quality Intelligence Platform

Interview & Portfolio Engineering Brief

End-to-end air-quality data platform for Lahore, Pakistan

Ingestion · Warehouse · AQI · Forecasting · API · Dashboard

Live dashboard: https://lahoreairquality.vercel.app

Source: github.com/alirazach476/lahoreairquality

This document describes implemented system behaviour and reproducible commands. Operational volume statistics that are not yet measured appear only as labelled placeholders. Model metrics below are taken from written evaluation artifacts in the repository (not invented for this brief).

1. Project overview

The Lahore Air Quality Intelligence Platform is a production-style portfolio system that continuously collects Lahore air-quality and weather observations, archives raw payloads, loads a PostgreSQL warehouse, transforms data with dbt, calculates United States EPA Air Quality Index (AQI) values, trains and serves short-horizon forecasts, and exposes results through a FastAPI service and a React dashboard.

Internal storage and computation use **UTC**. User-facing timestamps are displayed in **Asia/Karachi (PKT)**. The platform does not fabricate sensor readings or API responses.

2. Problem statement

Lahore experiences episodic severe particulate pollution. Reliable decision support requires more than a single public widget: raw feeds must be archived, quality-checked, joined with weather context, converted to a documented AQI standard, and forecasted with reproducible training and evaluation. Upstream monitors (for example AirNow via OpenAQ) already report US EPA AQI; this platform aligns calculation and serving to that standard while making ingestion failures, stale data, and model provenance explicit.

- Unify discovery and polling of OpenAQ monitors near Lahore with Open-Meteo weather.
- Guarantee raw archive + warehouse durability without silent data loss.
- Classify quality issues instead of deleting source rows.
- Serve current AQI, history, forecasts, sensors, and quality status via a documented API and UI.

3. Key features

- Runtime discovery of OpenAQ locations near configurable Lahore coordinates (no hard-coded station IDs).
- Kafka-compatible streaming (Redpanda) between poller and consumer with idempotent sinks.
- Dual-write path: object store (raw JSON) then PostgreSQL upserts.
- dbt staging → intermediate → marts, including AQI sub-index and feature marts.
- US EPA AQI from shared Python + dbt breakpoint tables (Feb 2024 PM2.5 breakpoints).
- Chronological ML evaluation with naive / seasonal-naive baselines, SARIMA, and XGBoost.
- MLflow experiment tracking for training runs.
- Dagster orchestration of pipeline assets and schedules.
- FastAPI with OpenAPI docs; React dashboard (Vercel-hosted frontend).
- Documented failure simulations for ten dependency / data-path scenarios.

4. Architecture

```
External APIs (OpenAQ v3, Open-Meteo)
  → ingestion/poller → Redpanda topics (aq.raw, weather.raw)
  → ingestion/consumer → object store (raw JSON) + PostgreSQL (raw)
  → dbt (staging → intermediate → marts; quality classify + report)
  → ml (features → train / infer; MLflow tracking)
  → apps/api (FastAPI) → apps/dashboard (React)
Dagster schedules and observes assets; it does not replace domain packages.
```

Design emphasis: partial failure isolation (AQ and weather poll independently), at-least-once delivery with idempotent sinks, S3-before-Postgres ordering, and observability of stale / missing / delayed data rather than silent imputation.

5. Tech stack

Layer	Technology
Ingestion	Python poller & consumer; Redpanda (Kafka API)
Archive	S3-compatible object storage (Adobe S3Mock locally)

Layer	Technology
Warehouse	PostgreSQL + Alembic migrations
Transforms	dbt
Orchestration	Dagster
ML / tracking	XGBoost, SARIMA (statsmodels), MLflow
API	FastAPI (OpenAPI)
Dashboard	React 19 + Vite + TypeScript
Local runtime	Docker Compose
CI	GitHub Actions
Frontend host	Vercel (static dashboard)

6. Data sources

Air quality — OpenAQ API v3

- Base URL via `OPENAQ_BASE_URL` (default https://api.openaq.org).
- Auth: header `X-API-Key` from `OPENAQ_API_KEY`.
- Discover monitors: `GET /v3/locations` with Lahore coordinates, radius, ISO=PK.
- Latest / historical measurements via location and sensor endpoints.
- Published free-tier rate limits (OpenAQ docs): 60 requests/min, 2,000/hour — subject to provider policy.

Weather — Open-Meteo

- Forecast/current: `OPENMETEO_BASE_URL`; archive: `OPENMETEO_ARCHIVE_BASE_URL`.
- No API key required for public endpoints used here.
- ML weather variables: temperature_2m, relative_humidity_2m, precipitation, wind_speed_10m (m/s), wind_direction_10m, surface_pressure.

Placeholder (not a measured result): After sustained production polling, report approximate volume as **~X readings/day** across **~N sensors** over **~Y months**. Do not substitute invented integers here.

7. Data pipeline

- **Poll:** scheduled OpenAQ + Open-Meteo fetches; publish normalized envelopes to Redpanda.
- **Consume:** validate → write deterministic object-store key → Postgres upsert → commit offset.
- **Transform:** dbt staging, intermediate (dedupe, normalize, AQI sub-index, weather join), marts.
- **Quality:** classify valid / suspect / invalid; retain rows; roll up report mart.
- **Features / train:** causal feature dataset; chronological splits; train SARIMA / XGBoost.
- **Serve:** FastAPI reads marts / forecast tables; dashboard polls JSON endpoints.

```
# Full local stack
cp .env.example .env
docker compose up -d --build

# dbt (Compose tooling profile)
make compose-dbt && make compose-dbt-test

# Optional history bootstrap (real providers; requires keys / network)
# see docs/bootstrap-history.md
```

8. Data quality

Suspicious and invalid observations are **classified and retained**. The platform never automatically deletes source rows based on quality rules. Disposition classes: **valid** | **suspect** | **invalid**.

Example rule	Severity	Condition (summary)
missing_timestamp	invalid	observed_at is null
missing_value	invalid	concentration missing / non-numeric
future_timestamp	invalid / suspect	beyond UTC now (+1h invalid; +5m–1h suspect)
unexpected_unit	invalid	unit not allowed for pollutant
negative_concentration	invalid	value < 0
duplicate_observation	suspect	natural key already present (row kept)
delayed_ingest	suspect	ingested_at – observed_at > 6 hours
missing_weather	suspect	no weather row for same UTC hour

```
python -m lahore_air_quality.quality
python -m lahore_air_quality.quality --json
# API: GET /api/data-quality
```

9. AQI methodology

Standard: **US EPA Air Quality Index** (40 CFR Part 58 Appendix G; EPA February 2024 PM2.5 breakpoint update effective 6 May 2024; AirNow TAD truncation rules). Breakpoints are copied from EPA tables into Python (`src/lahore_air_quality/aqi/`) and the dbt seed `aqi_breakpoints.csv` — never invented in code.

Rationale for Lahore portfolio use: primary OpenAQ coverage used in development (e.g. AirNow feeds) reports US EPA AQI. This is a documented engineering choice for consistency with upstream monitors, not a claim that EPA AQI is Pakistan's official national index.

- Concentrations are truncated (not rounded) per EPA TAD before breakpoint lookup.
- If the source unit does not match the EPA table unit for that pollutant, no AQI is calculated for that reading (no invented unit conversion).
- Overall AQI is the maximum sub-index among pollutants that could be scored.

10. Machine learning

Forecasting targets short-horizon PM2.5 / AQI-related labels on a causal feature dataset (lags, rolling statistics, weather, calendar). Splits are chronological (default 70% / 15% / 15%); series are never randomly shuffled. Feature construction forbids future leakage into training rows.

```
pip install -e "./ml[dev]"
python ml/training/train_xgboost.py \
  --from-features ml/features/datasets/lahore_aqi_features.parquet \
  --output-dir ml/training/artifacts/xgboost \
  --run-name xgboost -v
```

11. Model comparison

The evaluation package scores MAE, RMSE, and MAPE (MAPE excludes near-zero actuals). Baselines: naive last value and seasonal naive (default season 24 for hourly). The table below is copied from the repository artifact `ml/training/artifacts/xgboost/xgboost_metrics.md` (target `target_pm25_lead_1`, horizon 1, **test** split). These figures reflect that artifact's measured rows (**n_obs = 35** for XGBoost/naive; 34 for SARIMA reference) — not invented interview numbers. Scope is limited to the feature dataset used for that run; do not generalise as city-wide production accuracy without re-evaluating on a longer live window.

Model	MAE	RMSE	MAPE (fraction)	n_obs
xgboost_h1	2.8099	3.2844	0.1450 (~14.5%)	35
naive_last_value_h1	6.0179	7.0237	0.2573	35
seasonal_naive_s24	3.4664	4.1134	0.1797	35

Model	MAE	RMSE	MAPE (fraction)	n_obs
sarima_reference	4.1800	4.9832	0.1955	34

Placeholder (not a measured result): For a future production scorecard over a longer window, report test MAPE as **Z% MAPE** with explicit n_obs, date range (~Y months), and sensor coverage (~N sensors).

12. API

Method	Path	Purpose
GET	/health	Liveness + database probe
GET	/api/air-quality/current	Latest city/station AQI + weather + freshness
GET	/api/air-quality/history	Historical points (hours window)
GET	/api/forecast	Published model forecasts
GET	/api/sensors	Station inventory
GET	/api/data-quality	Quality snapshot
GET	/resilience/scenarios	Failure-scenario catalog
GET	/resilience/status	Live resilience probes

```
pip install -e "./apps/api[dev]"
lahore-aqi-api
# OpenAPI UI: http://localhost:8000/docs
```

13. Dashboard

React dashboard consumes only the public FastAPI origin (`VITE_API_BASE_URL`). UI includes an AQI gauge, Lahore time/place card, PM2.5 and weather metrics (PM10 omitted from UI), history (AQI / PM2.5 toggle), 24–48h forecast chart, pollutant bars, and sensors / quality panels. Empty, loading, network, and stale states are handled per panel.

```
cd apps/dashboard
cp .env.example .env   # VITE_API_BASE_URL=http://localhost:8000
npm install && npm run dev
# Production static host: https://lahoreairquality.vercel.app
```

14. Local setup

Prerequisites: Docker Engine + Compose v2; Node 22+ for Vite-only frontend work; Python 3.12+ for host tooling.

```
git clone https://github.com/alirazach476/lahoreairquality.git
cd lahoreairquality
cp .env.example .env
# Set OPENAQ_API_KEY for live air-quality polling

docker compose up -d --build
docker compose ps
docker compose logs -f api

# Host Python (optional)
pip install -e ".[dev]"
pip install -e apps/api -e ingestion/poller -e ingestion/consumer -e ml -e orchestration/
dagster
```

Service	URL
Dashboard	http://localhost:8080
API / OpenAPI	http://localhost:8000/docs
Dagster	http://localhost:3000

Service	URL
MLflow	http://localhost:5000

15. Environment variables

Copy `.env.example` → `.env`. Never commit secrets. Compose injects container service hostnames; host tools keep localhost.

Variable	Role
OPENAQ_API_KEY	OpenAQ v3 authentication
OPENAQ_BASE_URL / OPENMETEO_*	Provider hosts
LAHORE_LATITUDE / LONGITUDE	Discovery centre (WGS84)
POLL_INTERVAL_SECONDS	Poller cadence (default 300)
POSTGRES_*	Warehouse connection
REDPANDA_BROKERS / topics	Kafka-compatible broker + topics
S3_ENDPOINT_URL / S3_BUCKET / AWS_*	Object-store archive
MLFLOW_TRACKING_URI	Experiment tracking URI
API_CORS_ORIGINS	Allowed browser origins
VITE_API_BASE_URL	Dashboard build-time API origin

16. Docker setup

```
docker compose up -d --build          # or: make compose-up
docker compose down                   # keep volumes
docker compose down -v                # wipe volumes
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
docker compose --profile tools run --rm test
docker compose --profile tools run --rm dbt
make compose-train-synthetic          # XGBoost → MLflow (Compose tooling)
```

App images run as non-root (uid 10001) where practical. Health checks gate `depends_on` for Postgres, Redpanda, API, dashboard, Dagster, and MLflow.

17. Testing

```
# Default suite — mocked HTTP; no live provider credentials required
pip install -e ".[dev]" -e "ingestion/poller" -e "ingestion/consumer" -e "apps/api"
pytest
# or: make test

pytest tests/resilience -v            # failure simulations
# or: make test-resilience

RUN_EXTERNAL_API_TESTS=1 make test-integration   # opt-in live APIs
make smoke                            # e2e ingestion with mocked providers
```

External client scenario tests cover success, empty, invalid, HTTP failure, rate-limit, timeout, and retry behaviour for OpenAQ and Open-Meteo under mocks.

18. MLflow

- Tracking URI resolution: CLI → `MLFLOW_TRACKING_URI` → local `./mlruns`.
- Experiments: `lahore-aqi/xgboost`, `lahore-aqi/sarima`.
- Logged: parameters, MAE/RMSE/MAPE, training duration, model + prediction + metric artifacts.

- Disable per run with `--no-mlflow`.
- UI via Compose: http://localhost:5000

19. Failure handling

Principles: detect (typed errors, health probes), log structured events, retry when appropriate, never invent observations, recover on the next successful cycle / Kafka redelivery, expose status via metrics and HTTP. There is no dead-letter topic today: poison messages are rejected with offset commit; retryable sink failures hold the offset.

#	Scenario	Core behaviour
1	OpenAQ unavailable	AQ leg fails; Open-Meteo continues; next poll retries
2	Open-Meteo unavailable	Symmetric isolation for weather leg
3	Redpanda unavailable	No unpublished sink writes; producer/consumer retries
4	PostgreSQL unavailable	Offset held; S3 may already hold payload; API /health 503
5	Object store unavailable	No Postgres write without archive; offset held
6	Malformed event	Reject + commit (no fake readings)
7	Duplicate event	Idempotent upsert / deterministic S3 key
8	Stale sensor	Observability flags; freshness on API
9	Missing readings	Empty poll logged; gaps reported; no zero-imputation
10	Delayed event	suspect delayed_ingest; row retained

Details: `docs/failure-recovery.md`.

20. Project structure

```
apps/            FastAPI + React dashboard
data/            Local raw/processed placeholders
dbt/             Warehouse transforms and tests
ingestion/       Poller (produce) + consumer (archive + load)
ml/              Features, training, evaluation, inference
orchestration/   Dagster assets, jobs, schedules
infrastructure/  Dockerfiles, nginx, Postgres init
tests/           Unit / integration / resilience / smoke
docs/            Architecture, runbooks, interview PDF
scripts/         Dev utilities (e.g. seed_demo_forecasts)
src/             Shared AQI, DB, quality, resilience libraries
```

21. Screenshots

Replace or supplement the placeholders below with current captures from the running dashboard (local `http://localhost:8080` or lahoreairquality.vercel.app). Repository images live under `docs/images/`.

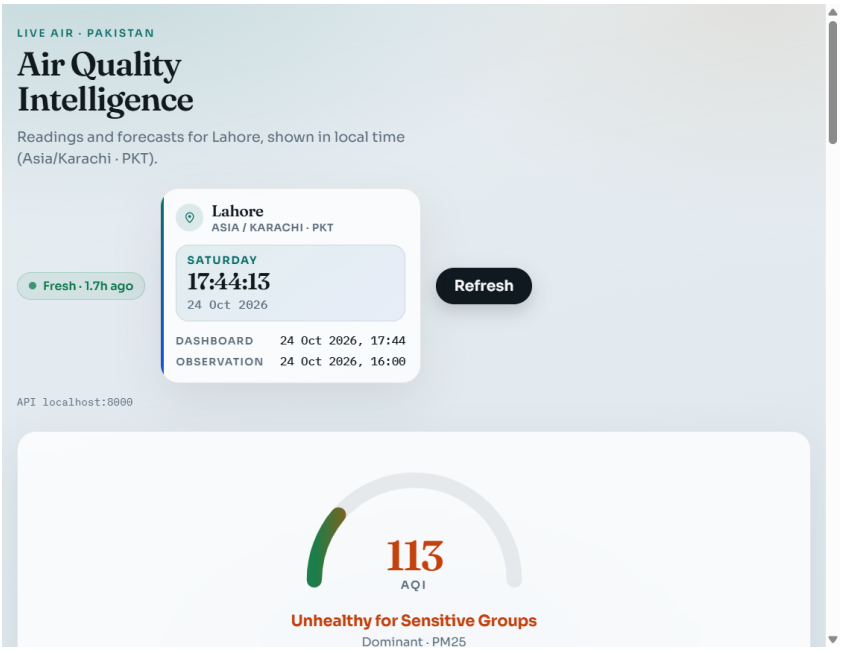


Figure 1. Dashboard hero — AQI gauge, Lahore time/place, freshness.



Figure 2. Historical trend and 24–48h forecast charts.

Placeholder (not a measured result): [Optional] Figure 3. MLflow UI experiment run · [Optional] Figure 4. Dagster asset graph · [Optional] Figure 5. OpenAPI /docs screenshot

22. Future improvements

- Host FastAPI + warehouse on a durable public backend so the Vercel UI always has a stable API origin.
- Named Cloudflare / reverse-proxy tunnel or cloud deployment instead of ephemeral quick tunnels.
- Dead-letter topic for poison messages with operator replay tooling.
- Longer live evaluation windows and station-level models with published scorecards.
- Optional unit conversion layer (documented) where OpenAQ units differ from EPA tables.
- Alerting on stale sensors and quality-class spikes.

23. Limitations

- Vercel hosts the static dashboard only; live charts require a reachable FastAPI backend and CORS allow-list.
- AQI is not calculated when upstream units do not match EPA table units (no silent conversion).
- XGBoost metrics above are from a finite feature-artifact test set (n_obs ≈ 35); not a multi-year citywide claim.
- No DLQ yet; rejected poison events are logged and committed.
- Local object store uses S3Mock in Compose — behaviour may differ from production S3/MinIO.
- Operational throughput (~X readings/day, ~N sensors, ~Y months) should be measured from warehouse queries before citing in interviews.

24. Author

Ali Raza Chaudhry
GitHub: github.com/alirazach476
Repository: github.com/alirazach476/lahoreairquality
Live dashboard: lahoreairquality.vercel.app

Document generated from repository documentation and measured ML artifacts. Regenerate with: `python docs/interview/generate_interview_pdf.py`